# Báo cáo tuần

## 1. Tổng quan

- Đoạn code xây dựng một pipeline huấn luyện hoàn chỉnh cho mô hình YOLOv3 . Toàn bộ hệ thống bao gồm các bước từ chuẩn bị dữ liệu, huấn luyện, đánh giá cho đến lưu mô hình, giúp đảm bảo quá trình training diễn ra liên tục và có thể khôi phục khi cần.

## 2. Nhóm hàm lưu và tải mô hình

- Hàm save_checkpoint
  Hàm này dùng để lưu toàn bộ trạng thái của quá trình huấn luyện tại một thời điểm cụ thể. Không chỉ lưu trọng số của mô hình mà còn lưu cả optimizer, scheduler, scaler và số epoch hiện tại. Điều này rất quan trọng vì cho phép tiếp tục huấn luyện từ đúng trạng thái trước đó khi bị gián đoạn.

```python
def save_checkpoint(model, optimizer, scheduler, scaler, epoch, filename):
    checkpoint = {
        "model": model.state_dict(),
        "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict(),
        "scaler": scaler.state_dict(),
        "epoch": epoch,
    }
    torch.save(checkpoint, filename)
    print(f"Checkpoint saved to {filename}.")
```

- Hàm save_model
  Khác với save_checkpoint, hàm này chỉ lưu trọng số của mô hình. Thường được sử dụng để lưu phiên bản mô hình tốt nhất sau khi đánh giá trên tập validation, phục vụ cho quá trình inference

```python
def save_model(model, filename):
    torch.save(model.state_dict(), filename)
    print(f"Model saved to {filename}.")
```

- Hàm load_checkpoint
  Hàm này thực hiện việc tải lại toàn bộ trạng thái đã lưu từ checkpoint. Sau khi load, mô hình và các thành phần liên quan sẽ trở về đúng trạng thái trước đó, giúp tiếp tục huấn luyện mà không cần bắt đầu lại từ đầu.

```python
def load_checkpoint(filename, model, optimizer, scheduler, scaler):
    print("Loading checkpoint...")
    checkpoint = torch.load(filename, map_location=config.DEVICE)

    model.load_state_dict(checkpoint["model"])
    optimizer.load_state_dict(checkpoint["optimizer"])

    if scheduler is not None:
        scheduler.load_state_dict(checkpoint["scheduler"])

    if scaler is not None:
        scaler.load_state_dict(checkpoint["scaler"])

    start_epoch = checkpoint["epoch"] + 1

    print("Checkpoint loaded successfully.")
    return start_epoch
```

### 3. Hàm train_fn

- Chức năng chính
  Hàm train_fn thực hiện toàn bộ quá trình huấn luyện mô hình trong một epoch. Đây là nơi diễn ra việc cập nhật trọng số dựa trên dữ liệu huấn luyện.

- Cách hoạt động
  Mô hình được chuyển sang chế độ train để đảm bảo các thành phần như batch normalization hoạt động đúng. Sau đó, dữ liệu từ train_loader được đưa vào theo từng batch. Với mỗi batch, dữ liệu được chuyển lên thiết bị tính toán, sau đó mô hình thực hiện lan truyền thuận để tạo dự đoán.

- Các tham số đầu vào

  **Train_loader**: Là DataLoader chứa dữ liệu huấn luyện, cung cấp các batch gồm ảnh và nhãn để đưa vào mô hình trong quá trình training.
  **Model**: Là mô hình YOLOv3 cần được huấn luyện, thực hiện việc dự đoán đầu ra từ dữ liệu đầu vào.
  **Optimizer**: Là thuật toán tối ưu dùng để cập nhật trọng số của mô hình dựa trên gradient, ví dụ như SGD.
  **Loss_fn**: Là hàm tính loss cho bài toán detection, trả về loss tổng và các thành phần như box, object, no object và class.
  **Scaler**: Là công cụ hỗ trợ mixed precision training, giúp tăng tốc tính toán và giảm sử dụng bộ nhớ GPU.
  **Epoch**:Là số thứ tự của epoch hiện tại, dùng để theo dõi tiến trình huấn luyện và tính toán learning rate.
  **Scheduler**:Là bộ điều chỉnh learning rate theo thời gian, giúp tối ưu quá trình hội tụ của mô hình, có thể có hoặc không.

- Tính toán loss
  Hàm loss trả về nhiều thành phần khác nhau bao gồm loss tổng, loss định vị bounding box, loss object, loss no object và loss phân loại. Việc tách riêng các thành phần này giúp theo dõi chi tiết quá trình học của mô hình.

```
running_loss += loss.item()
running_box += loss_box.item()
running_obj += loss_obj.item()
running_noobj += loss_noobj.item()
running_class += loss_class.item()
num_batches += 1
```

- Cập nhật trọng số
  Sau khi tính loss, quá trình lan truyền ngược được thực hiện để cập nhật trọng số. Kỹ thuật mixed precision được sử dụng thông qua autocast và GradScaler nhằm tăng tốc độ tính toán và giảm tiêu thụ bộ nhớ.

```
with torch.cuda.amp.autocast():
    out = model(x)
    loss, loss_box, loss_obj, loss_noobj, loss_class = loss_fn(out, targets)

    if torch.isnan(loss):
        print('Loss is becoming nan. Exiting')
        exit(0)

scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

- Điều chỉnh learning rate
  Ở giai đoạn đầu, cơ chế warmup được áp dụng để tăng dần learning rate từ giá trị nhỏ lên giá trị ban đầu, giúp mô hình học ổn định hơn. Sau đó, scheduler sẽ tiếp tục điều chỉnh learning rate theo chiến lược cosine annealing.

```
    # warmup / scheduler
    if global_step < warmup_iters:
        warmup_factor = global_step / warmup_iters
        lr = 1e-6 + base_lr * warmup_factor
        for param_group in optimizer.param_groups:
            param_group["lr"] = lr
    else:
        if scheduler is not None:
            scheduler.step()
```

- Kết quả trả về
  Sau khi hoàn thành toàn bộ batch, hàm trả về giá trị trung bình của các loại loss, phản ánh chất lượng huấn luyện trong epoch đó.

```
mean_loss = running_loss / num_batches
mean_box = running_box / num_batches
mean_obj = running_obj / num_batches
mean_noobj = running_noobj / num_batches
mean_class = running_class / num_batches

return mean_loss, mean_box, mean_obj, mean_noobj, mean_class
```

**4. Hàm eval_fn**

- Chức năng chính
  Hàm eval_fn dùng để đánh giá mô hình trên tập validation nhằm kiểm tra khả năng tổng quát hóa.

```
@torch.no_grad()
def eval_fn(
    val_loader,
    model,
    loss_fn,
):
    model.eval()
    print("Evaluating...")
```

- Cách hoạt động
  Mô hình được chuyển sang chế độ evaluation và không thực hiện cập nhật gradient. Dữ

liệu validation được đưa qua mô hình để tính toán loss tương tự như trong quá trình huấn luyện.

```
for batch_idx, (x, targets) in enumerate(loop):
    x = x.to(config.DEVICE)
    targets = [t.to(config.DEVICE) for t in targets]

    with torch.cuda.amp.autocast():
        out = model(x)
        loss, loss_box, loss_obj, loss_noobj, loss_class = loss_fn(out, targets)

    running_loss += loss.item()
    running_box += loss_box.item()
    running_obj += loss_obj.item()
    running_noobj += loss_noobj.item()
    running_class += loss_class.item()
    num_batches += 1

    mean_loss = running_loss / num_batches
    mean_box = running_box / num_batches
    mean_obj = running_obj / num_batches
    mean_noobj = running_noobj / num_batches
    mean_class = running_class / num_batches

    loop.set_postfix({
        "loss": f"{mean_loss:.3f}",
        "box": f"{mean_box:.3f}",
        "obj": f"{mean_obj:.3f}",
        "noobj": f"{mean_noobj:.3f}",
        "cls": f"{mean_class:.3f}",
    })
```

- Ý nghĩa
  Kết quả từ hàm này giúp so sánh giữa train loss và validation loss, từ đó phát hiện hiện tượng overfitting hoặc underfitting.

```
mean_loss = running_loss / num_batches
mean_box = running_box / num_batches
mean_obj = running_obj / num_batches
mean_noobj = running_noobj / num_batches
mean_class = running_class / num_batches

return (
    mean_loss,
    mean_box,
    mean_obj,
    mean_noobj,
    mean_class,
)
```

## 5. Hàm main

- Khởi tạo mô hình
  Mô hình YOLOv3 được khởi tạo và đưa lên thiết bị tính toán. Đồng thời, số lượng tham số có thể huấn luyện cũng được tính toán để đánh giá độ phức tạp của mô hình.

```
model = YOLOv3(num_classes=config.NUM_CLASSES).to(config.DEVICE)
total_trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Total trainable parameters: {total_trainable_params}")
```

  Hình dưới là số lượng trọng số có thể huấn luyện khi dùng mô hình với 2 backbone(ở trạng thái được unfreeze hoàn toàn) lần lượt là ResNet50 và Darknet53

```
Total trainable parameters: 47303457
```

```
Total trainable parameters: 61626049
```

  Nhận xét: Khi sử dụng backbone Darknet-53 trong YOLOv3 gốc, tổng số tham số của toàn mô hình vào khoảng 61,6 triệu tham số. Trong khi đó, mô hình sử dụng ResNet50 backbone chỉ có khoảng 47,3 triệu tham số trainable, nhẹ hơn đáng kể.

- Chuẩn bị dữ liệu
  Dữ liệu huấn luyện và validation được load thông qua DataLoader, đảm bảo việc chia batch và shuffle dữ liệu.

```
train_loader, test_loader, _, _ = get_train_test_loader()
```

- Cấu hình huấn luyện
  Optimizer được sử dụng là SGD với momentum và weight decay nhằm tối ưu hiệu quả học. Hàm loss là YOLOLoss, kết hợp với GradScaler để hỗ trợ mixed precision. Scheduler cosine annealing được sử dụng để điều chỉnh learning rate theo thời gian.

```python
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=config.LEARNING_RATE,
    momentum=0.9,
    weight_decay=5e-4,
    nesterov=True
)

loss_fn = YOLOLoss()
scaler = torch.cuda.amp.GradScaler()

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=len(train_loader) * config.NUM_EPOCHS,
    eta_min=1e-6
)
```

- Tham số huấn luyện

  Nhóm tham số này dùng để cấu hình toàn bộ quá trình huấn luyện mô hình. Trong đó, DEVICE xác định sử dụng GPU hay CPU, NUM_WORKERS và PIN_MEMORY giúp tối ưu tốc độ load dữ liệu. Các tham số như BATCH_SIZE, IMAGE_SIZE và NUM_CLASSES liên quan trực tiếp đến dữ liệu đầu vào và bài toán. LEARNING_RATE và NUM_EPOCHS điều khiển quá trình học của mô hình, còn LOAD_MODEL và SAVE_MODEL quyết định việc có sử dụng lại hoặc lưu lại mô hình trong quá trình huấn luyện.

```python
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
NUM_WORKERS = 6
BATCH_SIZE = 16
IMAGE_SIZE = 416
NUM_CLASSES = 20
LEARNING_RATE = 3e-4
NUM_EPOCHS = 100
LOAD_MODEL = True
SAVE_MODEL = True
PIN_MEMORY = torch.cuda.is_available()
```

- Vòng lặp huấn luyện
  Trong mỗi epoch, chương trình gọi hàm train_fn để huấn luyện mô hình. Sau đó checkpoint có thể được lưu lại để đảm bảo an toàn dữ liệu.

```python
for epoch in range(start_epoch, config.NUM_EPOCHS):
    mean_loss, mean_box, mean_obj, mean_noobj, mean_class = train_fn(
        train_loader,
        model,
        optimizer,
        loss_fn,
        scaler,
        epoch,
        scheduler,
    )

    if config.SAVE_MODEL:
        save_checkpoint(
            model,
            optimizer,
            scheduler,
            scaler,
            epoch,
            config.CHECKPOINT_FILE
        )
```

- Fine tuning mô hình
  Trong quá trình huấn luyện, mô hình sử dụng ResNet50 làm backbone thay cho Darknet-53 nhằm tận dụng khả năng trích xuất đặc trưng mạnh mẽ từ một mạng đã được pretrained. Ở giai đoạn đầu, một phần backbone được đóng băng để giữ nguyên các đặc trưng cơ bản đã học, giúp mô hình ổn định và tránh làm thay đổi các trọng số quan trọng quá sớm.

- Cơ chế freeze backbone
  Cụ thể, các lớp đầu của backbone như conv1, bn1, layer1 và layer2 được đặt requires_grad bằng False, tức là không cập nhật trọng số trong quá trình lan truyền ngược. Những lớp này thường học các đặc trưng mức thấp như cạnh, góc và texture nên không cần huấn luyện lại.

```python
def freeze_backbone(self):
    for name, param in self.backbone.named_parameters():
        if any(layer in name for layer in ["conv1", "bn1", "layer1", "layer2"]):
            param.requires_grad = False
```

- Mở khóa để fine tuning
  Sau một số epoch nhất định (Epoch thứ 12), các lớp backbone sẽ được mở khóa hoàn toàn để cho phép cập nhật toàn bộ trọng số. Quá trình này giúp mô hình thích nghi tốt hơn với dữ liệu cụ thể của bài toán, từ đó cải thiện độ chính xác tổng thể.Đánh giá và lưu mô hình tốt nhất

```python
if epoch + 1 == 12:
    print("UNFREEZING BACKBONE!")
    for param in model.backbone.parameters():
        param.requires_grad = True
```

- Sau mỗi vài epoch, mô hình được đánh giá trên tập validation. Nếu kết quả tốt hơn trước đó, mô hình sẽ được lưu lại làm phiên bản tốt nhất.

```python
if epoch > 0 and (epoch+1) % 3 == 0:
    val_loss, val_box, val_obj, val_noobj, val_class = eval_fn(
        test_loader,
        model,
        loss_fn,
    )
    with open(train_log_file, "a") as f:
        f.write(f"{epoch+1}, {mean_loss:.4f}, {val_loss:.4f}\n")

    if val_loss < best_test_loss:
        best_test_loss = val_loss
        save_model(model, f"weights/best_weight.pt")
```

**6. Kết quả huấn luyện**

Hình dưới đây là kết quả huấn luyện sau 100 epochs đầu tiên được đánh giá định kỳ sau mỗi 3 epoch

training_log.txt

```
Epoch, Train Loss, Val Loss
3, 3.5824, 3.4569
6, 3.1344, 3.1248
9, 2.9351, 2.9833
12, 2.8218, 2.8594
15, 2.7077, 2.7862
18, 2.6115, 2.7274
21, 2.5397, 2.6838
24, 2.4956, 2.7084
27, 2.4254, 2.6452
30, 2.3874, 2.6364
33, 2.3354, 2.6256
36, 2.2906, 2.6315
39, 2.2498, 2.6249
42, 2.2038, 2.5964
45, 2.1790, 2.6100
48, 2.1388, 2.6197
51, 2.1211, 2.5658
54, 2.0720, 2.5813
57, 2.0540, 2.6084
60, 2.0382, 2.5972
63, 1.9957, 2.6107
66, 1.9525, 2.6022
69, 1.9413, 2.6048
72, 1.9389, 2.5830
75, 1.9147, 2.5805
78, 1.8952, 2.6139
81, 1.8726, 2.6207
84, 1.8612, 2.6049
87, 1.8498, 2.5954
90, 1.8500, 2.6177
93, 1.8321, 2.6098
96, 1.8201, 2.6221
99, 1.8378, 2.6047
```

- Kết quả huấn luyện
  Trong quá trình huấn luyện, mô hình được đánh giá định kỳ sau mỗi 3 epoch. Tại mỗi lần đánh giá, hai giá trị chính được ghi nhận là train loss và validation loss nhằm theo dõi khả năng học và tổng quát hóa của mô hình. Kết quả cho thấy train loss giảm đều từ khoảng 3.58 ở epoch 3 xuống còn khoảng 1.82 ở epoch 96, cho thấy mô hình học tốt trên tập huấn luyện. Validation loss cũng có xu hướng giảm từ 3.45 xuống khoảng 2.58 đến 2.62, tuy nhiên mức giảm chậm hơn và có dao động nhẹ ở các epoch sau.
- Nhận xét
  Nhìn chung, mô hình đang học ổn định khi cả train loss và validation loss đều giảm theo thời gian. Tuy nhiên, từ khoảng epoch 30 trở đi, validation loss bắt đầu dao động và không giảm rõ rệt như train loss, cho thấy mô hình có dấu hiệu overfitting nhẹ. Dù vậy, mức chênh lệch giữa train loss và validation loss không quá lớn, nên mô hình vẫn giữ được khả năng tổng quát tương đối tốt.